



University of Global Village (UGV), Barishal

Department of Computer Science and Engineering

Object Oriented Programming (Java)

Course Teacher

MD. MEHADI HASAN

Lecturer

Department of CSE

University of Global Village(UGV), Barishal

Introduction to Classes, Objects, Methods, Strings

Outline

- Class – Definition
- Class – Example
- Class – First Example: Two Class
- Class – Second Example: Two copies of Object
- Declaring Objects
- Assigning Object Reference Variables
- Declaring a Method
- Adding a Method to the Box Class
- Method using Scanner Class

Class - Definition

- ❑ A class is a template for an object, and an object is an instance of a class. A class is declared by use of the class keyword.

A simplified general form of a class definition is shown here:

```
class classname{  
    type instance-variable1;  
    type instance-variable2;  
    // ...  
    type instance-variableN;  
    type methodname1(parameter-list) {  
        // body of method  
    }  
    type methodname2(parameter-list) {  
        // body of method  
    }  
    // ...  
    Type methodnameN(parameter-list) {  
        // body of method  
    }  
}
```

Class - Definition

The data, or variables, defined within a class are called **instance variables**. Collectively, the methods and variables defined within a class are called **members** of the class.

Java classes **do not need to have a main()** method. You only specify one if that class is the **starting point** for your program. Further, some kinds of Java applications, such as **applets**, **don't require a main()** method at all.

Class - Example

Declaring a class named Box:

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```



instance variables

Creating a Box object:

```
Box mybox = new Box(); // create a Box object called mybox  
mybox.width = 100;
```



mybox will be an instance of Box.

To access these variables, use **dot (.) operator**. The dot operator links the name of the object with the name of an instance variable.

Class - First Example: Two Class

```
/* A program that uses the Box class.

    Call this file BoxDemo.java
*/
class Box {
    double width;
    double height;
    double depth;
}

// This class declares an object of type Box.
class BoxDemo {
    public static void main(String args[]) {
        Box mybox = new Box();
        double vol;

        // assign values to mybox's instance variables
        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;

        // compute volume of box
        vol = mybox.width * mybox.height * mybox.depth;

        System.out.println("Volume is " + vol);
    }
}
```



Volume is 3000.0

Class - Second Example: Two copies of Object

Each object has its own copies of the instance variables. This means that if you have two Box objects, each has its own copy of depth, width, and height. It is important to understand that changes to the instance variables of **one object have no effect on the instance variables of another.**

```
// This program declares two Box objects.

class Box {
    double width;
    double height;
    double depth;
}

class BoxDemo2 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        // assign values to mybox1's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;

        /* assign different values to mybox2's
           instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;

        // compute volume of first box
        vol = mybox1.width * mybox1.height * mybox1.depth;
        System.out.println("Volume is " + vol);

        // compute volume of second box
        vol = mybox2.width * mybox2.height * mybox2.depth;
        System.out.println("Volume is " + vol);
    }
}
```

Volume is 3000.0

Volume is 162.0

Declaring Objects

- Declaring Objects

```
Box mybox = new Box();
```

This statement combines the two steps:

```
Box mybox; // declare reference to object  
mybox = new Box(); // allocate a Box object
```

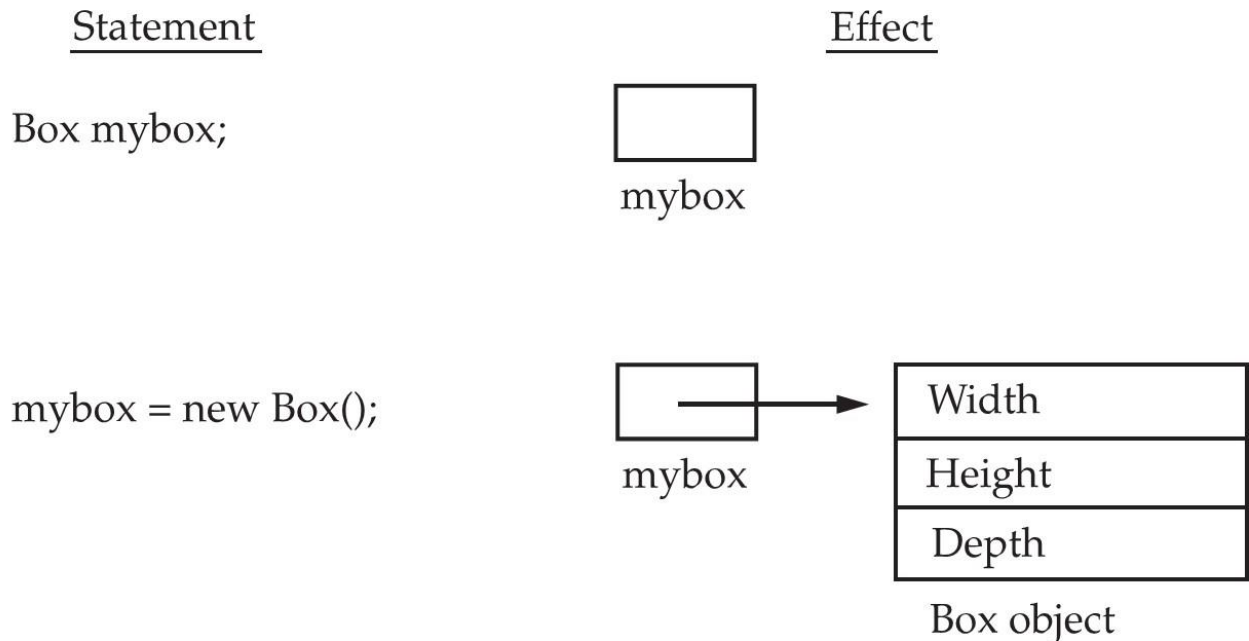
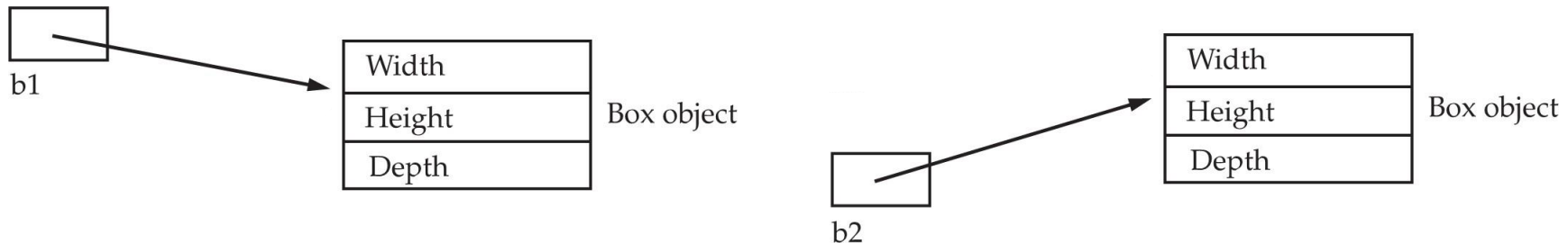


Figure 6-1 Declaring an object of type **Box**

Assigning Object Reference Variables

```
Box b1 = new Box();  
Box b2 = b1;
```

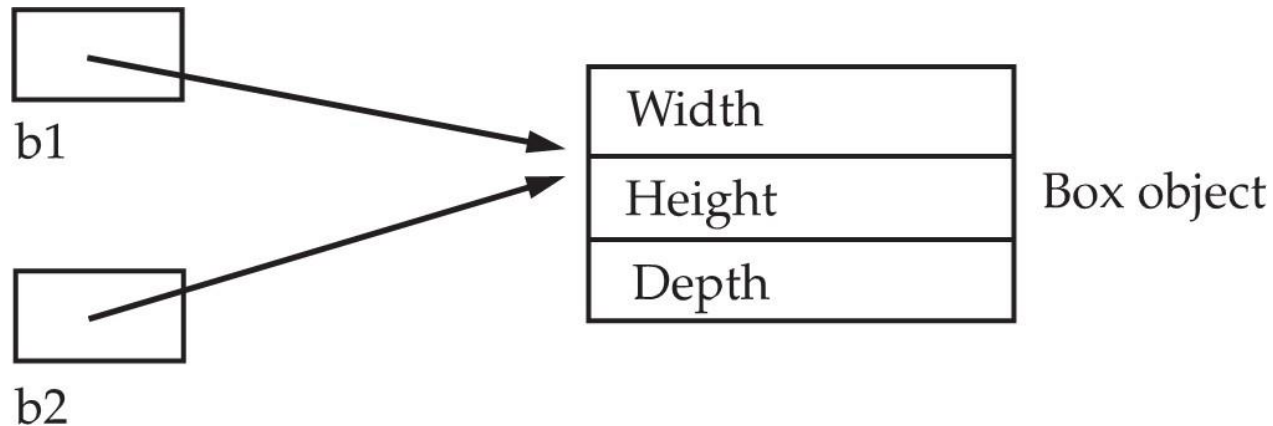
You might think that b2 is being assigned a reference to a **copy of the object referred to by b1**. That is, you might think that b1 and b2 refer to **separate and distinct objects**.



You are wrong.

Assigning Object Reference Variables

Actually, b1 and b2 will both refer to the same object.



Although b1 and b2 both refer to the same object, they are not linked in any other way.

```
Box b1 = new Box();  
Box b2 = b1;  
// ...  
b1 = null;
```

Here, **b1** has been set to **null**, but **b2** still points to the original object.

Declaring a Method

This is the general form of a method:

```
type name(parameter-list) {  
    // body of method  
}
```

Here, *type* specifies the type of data returned by the method. If the method does not return a value, its return type must be void.

Methods that have a return type other than void return a value to the calling routine using the following form of the return statement:

```
return value;
```

Adding a Method to the Box Class

```
// This program includes a method inside the box class.
```

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    // display volume of a box  
    void volume() {  
        System.out.print("Volume is ");  
        System.out.println(width * height * depth);  
    }  
}
```

```
class BoxDemo3 {  
    public static void main(String args[]) {
```

Adding a Method to the Box Class

```
Box mybox1 = new Box();
Box mybox2 = new Box();

// assign values to mybox1's instance variables
mybox1.width = 10;
mybox1.height = 20;
mybox1.depth = 15;

/* assign different values to mybox2's
   instance variables */
mybox2.width = 3;
mybox2.height = 6;
mybox2.depth = 9;

// display volume of first box
mybox1.volume();

// display volume of second box
mybox2.volume();
}
}
```

setMethods and getMethods

Classes often provide public methods to allow clients to set(i.e., assign values to) or get (i.e., obtain the values of) private instance variables.

```
1  // Fig. 3.7: GradeBook.java
2  // GradeBook class that contains a courseName instance variable
3  // and methods to set and get its value.
4
5  public class GradeBook
6  {
7      private String courseName; // course name for this GradeBook
8
9      // method to set the course name
10     public void setCourseName( String name )
11     {
12         courseName = name; // store the course name
13     } // end method setCourseName
14
15     // method to retrieve the course name
16     public String getCourseName()
17     {
18         return courseName;
19     } // end method getCourseName
20
21     // display a welcome message to the GradeBook user
22     public void displayMessage()
23     {
24         // calls getCourseName to get the name of
25         // the course this GradeBook represents
26         System.out.printf( "Welcome to the grade book for\n%s!\n",
27             getCourseName() );
28     } // end method displayMessage
29 } // end class GradeBook
```


setMethods and getMethods

```
1 // Fig. 3.8: GradeBookTest.java
2 // Creating and manipulating a GradeBook object.
3 import java.util.Scanner; // program uses Scanner
4
5 public class GradeBookTest
6 {
7     // main method begins program execution
8     public static void main( String[] args )
9     {
10         // create Scanner to obtain input from command window
11         Scanner input = new Scanner( System.in );
12
13         // create a GradeBook object and assign it to myGradeBook
14         GradeBook myGradeBook = new GradeBook();
15
16         // display initial value of courseName
17         System.out.printf( "Initial course name is: %s\n\n",
18             myGradeBook.getCourseName() );
19
20         // prompt for and read course name
21         System.out.println( "Please enter the course name:" );
22         String theName = input.nextLine(); // read a line of text
23         myGradeBook.setCourseName( theName ); // set the course name
24         System.out.println(); // outputs a blank line
25
26         // display welcome message after specifying course name
27         myGradeBook.displayMessage();
28     } // end main
29 } // end class GradeBookTest
```

Initial course name is: null
Please enter the course name:
CS101 Introduction to Java Programming
Welcome to the grade book for
CS101 Introduction to Java Programming!

Constructors

Constructor has the same name as the class in which it resides and is syntactically similar to a method. Once defined, the constructor is automatically called when the object is created, before the new operator completes. Constructors look a little strange because they have no return type, not even void.

Constructors

```
/* Here, Box uses a constructor to initialize the
   dimensions of a box.
*/
class Box {
    double width;
    double height;
    double depth;

    // This is the constructor for Box.
    Box() {
        System.out.println("Constructing Box");
        width = 10;
        height = 10;
        depth = 10;
    }

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}
```

Constructors

```
class BoxDemo6 {  
    public static void main(String args[]) {  
        // declare, allocate, and initialize Box objects  
        Box mybox1 = new Box();  
        Box mybox2 = new Box();  
  
        double vol;  
  
        // get volume of first box  
        vol = mybox1.volume();  
        System.out.println("Volume is " + vol);  
  
        // get volume of second box  
        vol = mybox2.volume();  
        System.out.println("Volume is " + vol);  
    }  
}
```

Parameterized Constructors

```
/* Here, Box uses a parameterized constructor to
   initialize the dimensions of a box.
*/
class Box {
    double width;
    double height;
    double depth;

    // This is the constructor for Box.
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}
```

Parameterized Constructors

```
class BoxDemo7 {  
    public static void main(String args[]) {  
        // declare, allocate, and initialize Box objects  
        Box mybox1 = new Box(10, 20, 15);  
        Box mybox2 = new Box(3, 6, 9);  
  
        double vol;  
  
        // get volume of first box  
        vol = mybox1.volume();  
        System.out.println("Volume is " + vol);  
  
        // get volume of second box  
        vol = mybox2.volume();  
        System.out.println("Volume is " + vol);  
    }  
}
```

this keyword

The this keyword can be used inside any method to refer to the current object.

The **this keyword** in Java refers to the **current object** of a class. It is used to access the current object's variables, methods, or constructors

```
// A redundant use of this.  
Box(double w, double h, double d) {  
    this.width = w;  
    this.height = h;  
    this.depth = d;  
}
```

this keyword

when a **local variable** has the same name as an **instance variable**, the local variable hides the instance variable. This is why width, height, and depth were not used as the names of the parameters to the Box() constructor inside the Boxclass.

We can use this keyword to resolve any namespace collisions that might occur between instance variables and local variables.

```
// Use this to resolve name-space collisions.  
Box(double width, double height, double depth) {  
    this.width = width;  
    this.height = height;  
    this.depth = depth;  
}
```

Without using this keyword

1. Differentiate Instance Variables from Local Variables

```
class Employee {  
    int id;  
    String name;  
  
    Employee(int id, String name) {  
        id = id;  
        name = name;  
    }  
  
    void display() {  
        System.out.println("ID: " + id);  
        System.out.println("Name: " + name);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Employee e1 = new Employee(101, "Hasan");  
        e1.display();  
    }  
}
```


Without using this keyword

Your Output

```
ID: 0  
Name: null
```

Explanation: Without using this

`id = id;` and `name = name;` assign the constructor parameters to themselves, so the object's variables remain unchanged. As a result, the object keeps its default values (0 and null).

Using this keyword

1. Differentiate Instance Variables from Local Variables

```
class Employee {  
    int id;  
    String name;  
  
    Employee(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println("ID: " + id);  
        System.out.println("Name: " + name);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Employee e1 = new Employee(101, "Hasan");  
        e1.display();  
    }  
}
```

Using this keyword

Your Output

ID: 101

Name: Hasan

Explanation: Using this

`this.id = id;` and `this.name = name;` assign the constructor parameters to the current object's instance variables. As a result, the object stores the values passed during object creation.